Robotic Navigation and Exploration - HW1

簡誌加 N26140692

演算法實作解析：A* 與 RRT*

這份簡報將帶您深入了解兩種經典的路徑規劃演算法：

- *A (A-Star) 演算法**：基於網格的啟發式搜尋，保證找到網格上的最佳解。
- *RRT (Rapidly-exploring Random Tree Star)**：基於隨機採樣的樹狀探索，適合大範圍連續空間，並具備重新接枝（Rewire）的最佳化能力。

接下來我們將拆解程式碼，一步步解析演算法的運行邏輯！

A* 演算法：核心概念

A* 的核心在於它的評估函數： $f(n) = g(n) + h(n)$

- $g(n)$：從起點走到當前節點的**實際花費**。
- $h(n)$：從當前節點走到終點的**預估花費**（Heuristic，此處使用直線距離）。
- 演算法每次都會優先挑選 $f(n)$ 最小的節點進行展開，兼顧了「走過的路徑最短」與「朝向終點前進」兩個目標。

A* 步驟拆解一：初始化（`preloop`）

在開始尋找路徑前，我們需要準備兩個重要的集合與紀錄花費的字典：

- `open_set`：待探索的節點（一開始只有起點）。
- `closed_set`：已經探索完畢的節點。

```
def preloop(self):
    # 1. 初始化資料結構
    self.open_set: set[PathNode] = {self.start_node}
    self.closed_set: set[PathNode] = set()
    self.visited_nodes: set[PathNode] = {self.start_node}

    # 2. 初始化 g 值與 h 值
    self.g: dict[PathNode, float] = {self.start_node: 0}
    self.h: dict[PathNode, float] = {
        self.start_node: calculate_node_distance(self.start_node, self.goal_node)
    }
```

A* 步驟拆解二：節點選擇與終點判定 (`step`)

每次迴圈，我們從 `open_set` 中挑出 f 值最小的節點。這裡使用了一個小技巧 **Tie-breaking**：當 f 值相同時，優先選擇 g 值較大（離起點較遠）的節點，可大幅減少不必要的平行展開。

```
# 從 open_set 找出 f=g+h 最小的節點
current_node = min(
    self.open_set,
    key=lambda node: (
        self.g.get(node, float('inf')) + self.h.get(node, float('inf')),
        -self.g.get(node, 0) # 加上負號讓 g 越大的排越前面
    )
)
# 檢查是否抵達終點 (小於目標容許值 threshold)
dist_to_goal = calculate_node_distance(current_node, self.goal_node)
if dist_to_goal <= self.goal_threshold:
    self.goal_node.parent = current_node # 接上終點
    self.is_done.set()
    return
```

A* 步驟拆解三：展開鄰居節點 (`step`)

確定踩上 `current_node` 後，將其移至 `closed_set` ，並觀察周圍合法的鄰居。如果找到了一條更短的路徑前往鄰居（ `tentative_g` 較小），就更新它的父節點與花費。

```
self.open_set.remove(current_node)
self.closed_set.add(current_node)
for neighbor in self.get_neighbor_nodes(current_node):
    if neighbor in self.closed_set: continue
    move_cost = calculate_node_distance(current_node, neighbor)
    tentative_g = self.g[current_node] + move_cost
    # 如果是新鄰居，或是找到更短的路
    if neighbor not in self.open_set or tentative_g < self.g.get(neighbor, float('inf')):
        neighbor.parent = current_node # 記錄來源以便回溯
        self.g[neighbor] = tentative_g
        self.h[neighbor] = calculate_node_distance(neighbor, self.goal_node)
        self.open_set.add(neighbor)
```

RRT* 演算法：核心概念

RRT* 不依賴固定網格，而是透過「隨機生長」的方式探索空間。它包含以下核心步驟：

1. **Sample**：在空間中隨機採樣一個點。
2. **Steer**：從樹上最近的節點往採樣點走一小步，產生新節點。
3. **Choose Parent**：在半徑內尋找最佳（代價最低）的父節點。
4. **Rewire (重新佈線)**：這是 "Star" 的精髓！檢查新節點是否能成為附近節點更好的父節點，從而讓整體路徑更平滑、成本更低。

RRT* 步驟拆解一：採樣與控制生長 (Sample & Steer)

我們先在空間中隨機找一個點 `random_node`，接著找出樹上離它最近的點 `nearest_node`，並朝它的方向前進 `step_size` 的距離，計算出真正要加入的新節點座標。

```
# 1. 隨機採樣與尋找最近點
random_node = self.sample_random_node()
nearest_node = min(self.tree, key=lambda n: calculate_node_distance(n, random_node))
# 2. 幾何運算：往前走 step_size 的距離
dist_to_random = calculate_node_distance(nearest_node, random_node)
if dist_to_random <= self.step_size:
    new_x, new_y = random_node.coordinates.x, random_node.coordinates.y
else:
    dx = random_node.coordinates.x - nearest_node.coordinates.x
    dy = random_node.coordinates.y - nearest_node.coordinates.y
    new_x = nearest_node.coordinates.x + (dx / dist_to_random) * self.step_size
    new_y = nearest_node.coordinates.y + (dy / dist_to_random) * self.step_size

new_node = PathNode(coordinates=PixelCoordinates(new_x, new_y))
# (後續進行碰撞與邊界檢查...)
```

RRT* 步驟拆解二：選擇最佳父節點 (Choose Parent)

在 `search_radius` 的範圍內找出所有鄰近節點（`near_nodes`），並一一比較：「如果從這些節點連過來，誰的總代價最小？」藉此找到最佳的接枝點。

```
near_nodes = [
    n for n in self.tree
    if calculate_node_distance(n, new_node) <= self.search_radius
]

# 預設以 nearest_node 為父節點
min_cost = nearest_node.cost + calculate_node_distance(nearest_node, new_node)
best_parent = nearest_node

for near_node in near_nodes:
    potential_cost = near_node.cost + calculate_node_distance(near_node, new_node)
    # 如果成本更低且沒有撞牆，就替換最佳父節點
    if potential_cost < min_cost and check_collision_free(self.occupancy_map, near_node, new_node):
        min_cost = potential_cost
        best_parent = near_node

new_node.parent = best_parent
new_node.cost = min_cost
self.tree.append(new_node)
```

RRT* 步驟拆解三：重新接枝 (Rewire)

這是 RRT 得以逼近最佳解的關鍵！ * 新節點加入後，我們要反過來問：「附近的節點如果改認新節點當父節點，總代價會不會更便宜？」如果會，就把這根樹枝「剪斷重接」。

```
for near_node in near_nodes:
    # 試算：如果 near_node 改從 new_node 連過來的花費
    rewire_cost = new_node.cost + calculate_node_distance(new_node, near_node)

    if rewire_cost < near_node.cost and check_collision_free(self.occupancy_map, new_node, near_node):
        # 重新接枝 (Rewire)
        near_node.parent = new_node
        near_node.cost = rewire_cost

# (最後檢查 new_node 是否已經進入終點範圍 threshold 內, 若進入則結束)
```

總結

- **A*** 提供了嚴謹且具方向性的搜尋，透過網格與 Heuristic Function 穩定找出最佳路徑，但受限於網格的解析度。
- **RRT*** 利用隨機生長快速突破複雜障礙物，並透過 Choose Parent 與 Rewire 機制，讓原本曲折的隨機樹逐漸優化成平滑且低成本的最佳路徑。